

FIELD GUIDE · ARCHITECTURE & CODE

The Modern Web + AI Stack

An exhaustive, code-level tour of frontend, backend, data, AI integration (RAG / CAG / MAG), 3D & interaction tooling, security, and deployment — built for engineers shipping production AI products.

Prepared August 2026 · Includes companion source files (TypeScript routers, chunking pipelines, caching layers, rate limiters, auth middleware)

Table of Contents

- [01](#) The Frontend (UI Layer)
- [02](#) The Backend (API & Server Layer)
- [03](#) Data & Caching
- [04](#) AI Integration Architectures (RAG / CAG / MAG)
- [05](#) Security & Governance
- [06](#) 3D, Vision & Interaction Tooling
- [07](#) Voice & Real-Time Multimodal
- [08](#) DevOps, Deployment & Observability
- [09](#) API Keys, Tokens & Free Tiers
- [10](#) Reference Architecture
- [11](#) Companion Code Index
- [12](#) Pre-Launch Security Checklist
- [13](#) From Vibe-Coded to Production-Grade
- [14](#) Pre-Launch Website & Content Checklist
- [15](#) Beyond the Checklist: Deeper Production Maturity

01 · The Frontend (UI Layer)

The frontend is the negotiation layer between a human and a system. Every choice here trades off developer velocity, runtime performance, and bundle weight.

1.1 Languages

- **HTML** — semantic document structure; drives accessibility (ARIA) and SEO.
- **CSS** — visual presentation; modern layout via Flexbox/Grid, container queries.
- **JavaScript / TypeScript** — logic layer. TypeScript adds static types, which matters enormously once AI responses (dynamic, loosely-typed JSON) enter the client — type-safe parsing with `zod` or `io-ts` prevents whole classes of runtime crashes.

1.2 Frameworks

Framework	Rendering Model	Best Fit
React	Virtual DOM, component tree	Large ecosystems, AI SDK support (Vercel AI SDK is React-first)
Vue.js	Reactive proxies, SFCs	Gentler learning curve, strong DX
Svelte / SvelteKit	Compile-time reactivity, no VDOM	Small bundles, fast interactivity
Angular	Full MVC framework	Large enterprise teams, strict structure

1.3 Styling

Tailwind CSS is the dominant utility-first approach for AI-assisted development because its class names are predictable tokens an LLM can generate reliably. **Sass** remains relevant for design systems needing nested rules, mixins, and variables at scale.

1.4 State Management

Redux (Toolkit)

Predictable, centralized store with strict unidirectional data flow. Best for large teams needing time-travel debugging and strict conventions.

Zustand

Minimal, hook-based store with almost no boilerplate. Preferred in modern AI apps for holding chat state, streaming tokens, and UI state side by side.

Code — Zustand store for a streaming AI chat

```
// store/chatStore.ts
import { create } from 'zustand';

interface Message { id: string; role: 'user' | 'assistant'; content: string; }

interface ChatState {
  messages: Message[];
  isStreaming: boolean;
  appendToken: (id: string, token: string) => void;
  addMessage: (msg: Message) => void;
  setStreaming: (v: boolean) => void;
}

export const useChatStore = create<ChatState>((set) => ({
  messages: [],
  isStreaming: false,
  addMessage: (msg) => set((s) => ({ messages: [...s.messages, msg] })),
  appendToken: (id, token) =>
    set((s) => ({
      messages: s.messages.map((m) =>
        m.id === id ? { ...m, content: m.content + token } : m
      ),
    })),
  setStreaming: (v) => set({ isStreaming: v }),
}));
```

02 · The Backend (API & Server Layer)

2.1 Languages & Frameworks

Stack	Strength	Typical Use
Node.js + Express	Same language as frontend, huge npm ecosystem	General APIs, AI proxy/streaming layers
Python + FastAPI	Async-native, auto OpenAPI docs, best AI/ML library access	Model-serving, RAG pipelines, data workloads
Python + Django	Batteries-included, ORM, admin panel	Content-heavy apps, fast CRUD
Go	Concurrency, low memory, single binary	High-throughput gateways, edge services
Rust	Memory safety without GC, extreme performance	Vector search engines, latency-critical inference glue
Java + Spring Boot	Mature, strongly typed, enterprise tooling	Regulated / large enterprise backends

2.2 API Paradigms

REST

Resource-oriented, verbs map to HTTP methods (GET/POST/PUT/PATCH/DELETE). Simple caching semantics, ubiquitous tooling. Default choice for most AI product APIs.

GraphQL

Single endpoint, client specifies the exact shape of data needed. Avoids over/under-fetching; more complex caching and N+1 query risk.

WebSockets

Full-duplex, persistent connection. Required for token-by-token LLM streaming fallback, live presence, collaborative cursors, and chat.

Code — Express router with versioning, validation & streaming (TypeScript)

```
// src/routes/chat.router.ts
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { rateLimiter } from '../middleware/rateLimiter';
import { requireAuth } from '../middleware/auth';
import { streamCompletion } from '../lib/aiClient';

export const chatRouter = Router();

const ChatRequestSchema = z.object({
  message: z.string().min(1).max(4000),
  conversationId: z.string().uuid().optional(),
});

// POST /api/v1/chat — Server-Sent Events streaming response
chatRouter.post(
  '/v1/chat',
  requireAuth,
  rateLimiter({ windowSeconds: 60, max: 20 }),
  async (req: Request, res: Response) => {
    const parsed = ChatRequestSchema.safeParse(req.body);
    if (!parsed.success) {
      return res.status(422).json({ error: parsed.error.flatten() });
    }

    res.setHeader('Content-Type', 'text/event-stream');
    res.setHeader('Cache-Control', 'no-cache');
    res.setHeader('Connection', 'keep-alive');

    try {
      for await (const token of streamCompletion(parsed.data.message)) {
        res.write(`data: ${JSON.stringify({ token })}\n\n`);
      }
      res.write('data: [DONE]\n\n');
    } catch (err) {
      res.write(`data: ${JSON.stringify({ error: 'stream_failed' })}\n\n`);
    } finally {
      res.end();
    }
  }
);
```

Code — Central router registry (protocol map)

```
// src/routes/index.ts
import { Router } from 'express';
import { chatRouter } from '../chat.router';
import { ragRouter } from '../rag.router';
import { userRouter } from '../user.router';

export const apiRouter = Router();

/**
 * Route protocol table
 * -----
 * Prefix      Protocol   Auth      Rate-limited
 * /v1/chat    HTTP (SSE)  required  yes (20/min)
 * /v1/rag     HTTP       required  yes (10/min)
 * /v1/users   HTTP (REST) required  yes (60/min)
 * /ws         WebSocket  required  connection-based
 * -----
 */
apiRouter.use(chatRouter);
apiRouter.use(ragRouter);
apiRouter.use(userRouter);
```

Code — WebSocket server for real-time features

```
// src/ws/server.ts
import { WebSocketServer, WebSocket } from 'ws';
import { verifyToken } from '../lib/auth';

const HEARTBEAT_MS = 30_000;

export function createWSServer(server: import('http').Server) {
  const wss = new WebSocketServer({ server, path: '/ws' });

  wss.on('connection', async (socket: WebSocket, req) => {
    const token = new URL(req.url!, 'http://x').searchParams.get('token');
    const user = await verifyToken(token).catch(() => null);
    if (!user) return socket.close(4001, 'unauthorized');

    let alive = true;
    socket.on('pong', () => (alive = true));

    const heartbeat = setInterval(() => {
      if (!alive) return socket.terminate();
      alive = false;
      socket.ping();
    }, HEARTBEAT_MS);

    socket.on('message', (raw) => {
      const msg = JSON.parse(raw.toString());
      // route by msg.type: 'chat.token' | 'presence.update' | 'notify'
    });

    socket.on('close', () => clearInterval(heartbeat));
  });

  return wss;
}
```

03 · Data & Caching

3.1 Relational — PostgreSQL

Structured, ACID-compliant storage with strong relational integrity. The default choice for users, billing, permissions, and anything needing joins and transactions. Modern Postgres also supports vector search via the `pgvector` extension, letting you keep embeddings next to relational data instead of a separate vector database.

3.2 NoSQL — MongoDB

Document-oriented storage for flexible, evolving schemas — good for AI conversation logs, feature flags, and content that doesn't fit neatly into rows and columns.

3.3 Caching — Redis / Upstash

An in-memory key-value store. Beyond simple caching, Redis underpins rate limiting, session storage, pub/sub for real-time events, and job queues (via BullMQ). **Upstash** provides a serverless, HTTP-based Redis that works natively in edge runtimes (Vercel Edge, Cloudflare Workers) where a persistent TCP connection isn't available.

Code — Cache-aside pattern with Upstash Redis

```
// src/lib/cache.ts
import { Redis } from '@upstash/redis';

export const redis = Redis.fromEnv();

export async function cached<T>({
  key: string,
  ttlSeconds: number,
  fetcher: () => Promise<T>
}): Promise<T> {
  const hit = await redis.get<T>(key);
  if (hit !== null) return hit;

  const value = await fetcher();
  await redis.set(key, value, { ex: ttlSeconds });
  return value;
}

// Usage
// const profile = await cached(`user:${id}`, 300, () => db.user.findUnique({ where: { id } }));
```

04 · AI Integration Architectures

Connecting an LLM to a web app is fundamentally a context-management problem: the model only knows what's inside its context window at inference time. Three dominant patterns solve this.

Architecture	Core Mechanism	Best For	Speed
RAG (Retrieval-Augmented)	Fetch relevant chunks dynamically per query via vector/keyword search	Frequently changing data, internal knowledge bases	Moderate
CAG (Cache-Augmented)	Pre-load stable knowledge directly into the model's context/KV-cache	Static policies, FAQs, high-volume repetitive tasks	Very Fast
MAG (Memory-Augmented)	Persist and recall structured user history across sessions	AI companions, personalized assistants	Varies

4.1 RAG Pipeline & Chunking

Chunking strategy is the single highest-leverage decision in a RAG system — chunks too large dilute retrieval precision; chunks too small lose surrounding context. A common approach is recursive, token-aware splitting with overlap.

Code — Recursive token-aware chunking

```
// src/lib/chunking.ts
interface Chunk { id: string; text: string; tokens: number; metadata: Record<string, unknown>; }

const SEPARATORS = ['\n\n', '\n', '. ', ' ']; // paragraph -> line -> sentence -> word

export function chunkText(
  text: string,
  { maxTokens = 400, overlapTokens = 60, metadata = {} }: {
    maxTokens?: number; overlapTokens?: number; metadata?: Record<string, unknown> } = {}
): Chunk[] {
  const estimateTokens = (s: string) => Math.ceil(s.length / 4); // ~4 chars/token heuristic

  function split(input: string, sepIdx: number): string[] {
    if (sepIdx >= SEPARATORS.length) return [input];
    const parts = input.split(SEPARATORS[sepIdx]).filter(Boolean);
    return parts.length > 1 ? parts : split(input, sepIdx + 1);
  }

  const rawSegments = split(text, 0);
  const chunks: Chunk[] = [];
  let buffer: string[] = [];
  let bufferTokens = 0;

  const flush = () => {
    if (!buffer.length) return;
    const chunkStr = buffer.join(' ').trim();
    chunks.push({
      id: `chunk_${chunks.length}`,
      text: chunkStr,
      tokens: estimateTokens(chunkStr),
      metadata,
    });
  };

  for (const seg of rawSegments) {
    const segTokens = estimateTokens(seg);
    if (bufferTokens + segTokens > maxTokens) {
      flush();
      // carry overlap forward
      const overlapWords = buffer.join(' ').split(' ').slice(-overlapTokens);
      buffer = [overlapWords.join(' ')];
      bufferTokens = estimateTokens(buffer[0]);
    }
    buffer.push(seg);
    bufferTokens += segTokens;
  }
  flush();
  return chunks;
}
```

Code — Embedding + retrieval (pgvector)

```
// src/lib/rag.ts
import { chunkText } from './chunking';
import { db } from './db';
import { embed } from './aiClient';

export async function ingestDocument(docId: string, rawText: string) {
  const chunks = chunkText(rawText, { maxTokens: 400, overlapTokens: 60, metadata: { docId } });
  for (const chunk of chunks) {
    const vector = await embed(chunk.text); // number[] e.g. 1536-dim
    await db.query(
      `INSERT INTO document_chunks (id, doc_id, content, embedding)
      VALUES ($1, $2, $3, $4)`,
      [chunk.id, docId, chunk.text, vector]
    );
  }
}

export async function retrieveContext(query: string, k = 5) {
  const queryVector = await embed(query);
  const { rows } = await db.query(
    `SELECT content, doc_id, 1 - (embedding <=> $1) AS similarity
    FROM document_chunks
    ORDER BY embedding <=> $1
    LIMIT $2`,
    [queryVector, k]
  );
  return rows; // top-k most relevant chunks, injected into the prompt
}
```

4.2 CAG — Cache-Augmented Generation

Instead of retrieving at query time, CAG pre-computes and reuses the model's internal key-value cache for a fixed knowledge block (e.g. a company policy doc), then appends only the user's query. This trades flexibility for latency — ideal when the knowledge base rarely changes.

Rule of thumb: if your source documents change less than once a day and fit comfortably in context, CAG beats RAG on both latency and simplicity. If they change per-user or per-minute, use RAG.

4.3 MAG — Memory-Augmented Generation

Persists distilled facts about a user (preferences, past decisions, ongoing projects) in a structured store, retrieved and injected at the start of each new session — similar to Claude's own memory feature.

Code — Simple memory store schema + injection

```
// src/lib/memory.ts
interface MemoryFact { userId: string; key: string; value: string; updatedAt: Date; }

export async function upsertMemory(userId: string, key: string, value: string) {
  await db.query(
    `INSERT INTO user_memory (user_id, key, value, updated_at)
    VALUES ($1, $2, $3, now())
    ON CONFLICT (user_id, key) DO UPDATE SET value = $3, updated_at = now()`,
    [userId, key, value]
  );
}

export async function buildMemoryPreamble(userId: string): Promise<string> {
  const { rows } = await db.query<MemoryFact>(
    `SELECT key, value FROM user_memory WHERE user_id = $1 ORDER BY updated_at DESC LIMIT 20`,
    [userId]
  );
  if (!rows.length) return '';
  return 'Known context about this user:\n' + rows.map(r => ` - ${r.key}: ${r.value}`).join('\n');
}
```


05 · Security & Governance

5.1 Rate Limiting

Protects against abuse, scraping, and runaway AI-token costs. A sliding-window or token-bucket algorithm backed by Redis is standard; Upstash ships a ready-made `@upstash/ratelimit` package for edge environments.

Code — Sliding-window limiter (Express middleware)

```
// src/middleware/rateLimiter.ts
import { Ratelimit } from '@upstash/ratelimit';
import { redis } from '../lib/cache';
import { Request, Response, NextFunction } from 'express';

export function rateLimiter({ windowSeconds = 60, max = 30 }: { windowSeconds?: number; max?: number }) {
  const limiter = new Ratelimit({
    redis,
    limiter: Ratelimit.slidingWindow(max, `${windowSeconds} s`),
    analytics: true,
  });

  return async (req: Request, res: Response, next: NextFunction) => {
    const identifier = (req as any).user?.id ?? req.ip;
    const { success, remaining, reset } = await limiter.limit(identifier);

    res.setHeader('X-RateLimit-Remaining', remaining);
    if (!success) {
      return res.status(429).json({
        error: 'rate_limited',
        retryAfterMs: reset - Date.now(),
      });
    }
    next();
  };
}
```

5.2 Authentication

Verifies *who* the user is. Common patterns: signed JWTs for stateless API auth, or session cookies backed by Redis/Postgres. Supabase Auth and Clerk offload most of this entirely.

Code — JWT verification middleware

```
// src/middleware/auth.ts
import jwt from 'jsonwebtoken';
import { Request, Response, NextFunction } from 'express';

export function requireAuth(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'missing_token' });
  }
  try {
    const payload = jwt.verify(header.slice(7), process.env.JWT_SECRET!);
    (req as any).user = payload;
    next();
  } catch {
    return res.status(401).json({ error: 'invalid_token' });
  }
}
```

Never store raw API keys in frontend bundles. Route all third-party AI calls (OpenAI, Anthropic, etc.) through your backend or a serverless edge function so secrets stay server-side.

06 · 3D, Vision & Interaction Tooling

- Blender** Free
Full 3D content-creation suite — modeling, sculpting, rigging, rendering. Used to author source assets exported as glTF/GLB for the web.
- Spline** Free tier
Browser-based 3D design tool aimed at web designers; exports directly to React components or embeds via iframe/runtime.
- React Three Fiber**
A React renderer for **Three.js** — lets you build 3D scenes declaratively as React components, composing with the rest of your app's state (Zustand/Redux) instead of imperative Three.js code.
- MediaPipe (Hand/Body Landmarks)**
Google's on-device ML pipeline for real-time hand, face, and pose tracking from webcam input — enables gesture-controlled UI entirely client-side, no server round-trip.
- SAM (Segment Anything) / SAM 3D**
Meta's foundation model family for image and 3D segmentation — click-to-select objects in an image, or lift 2D detections into 3D scene understanding for AR/AI-assisted editing tools.

Code — React Three Fiber scene with MediaPipe hand-gesture control

```

// components/GestureScene.tsx
import { Canvas, useFrame } from '@react-three/fiber';
import { useRef, useEffect, useState } from 'react';
import { HandLandmarker, FilesetResolver } from '@mediapipe/tasks-vision';

function RotatingCube({ rotationSpeed }: { rotationSpeed: number }) {
  const meshRef = useRef<any>(null);
  useFrame(() => { if (meshRef.current) meshRef.current.rotation.y += rotationSpeed; });
  return (
    <mesh ref={meshRef}>
      <boxGeometry args={[1, 1, 1]} />
      <meshStandardMaterial color="#2563eb" />
    </mesh>
  );
}

export default function GestureScene() {
  const [speed, setSpeed] = useState(0.01);
  const videoRef = useRef<HTMLVideoElement>(null);

  useEffect(() => {
    let landmarker: HandLandmarker;
    (async () => {
      const vision = await FilesetResolver.forVisionTasks(
        'https://cdn.jsdelivr.net/npm/@mediapipe/tasks-vision/wasm'
      );
      landmarker = await HandLandmarker.createFromOptions(vision, {
        baseOptions: { modelAssetPath: '/models/hand_landmarker.task' },
        numHands: 1,
      });

      const stream = await navigator.mediaDevices.getUserMedia({ video: true });
      videoRef.current!.srcObject = stream;

      const loop = () => {
        if (videoRef.current) {
          const result = landmarker.detectForVideo(videoRef.current, performance.now());
          if (result.landmarks[0]) {
            const spread = handSpread(result.landmarks[0]); // 0..1
            setSpeed(0.005 + spread * 0.08);
          }
        }
        requestAnimationFrame(loop);
      };
      loop();
    })();
  }, []);

  return (
    <>
      <video ref={videoRef} autoPlay muted style={{ display: 'none' }} />
      <Canvas>
        <ambientLight intensity={0.6} />
        <pointLight position={[5, 5, 5]} />
        <RotatingCube rotationSpeed={speed} />
      </Canvas>
    </>
  );
}

function handSpread(landmarks: { x: number; y: number }[]) {
  const dx = landmarks[4].x - landmarks[20].x;
  const dy = landmarks[4].y - landmarks[20].y;
  return Math.min(1, Math.hypot(dx, dy) * 2);
}

```

07 · Voice & Real-Time Multimodal

Voice chat pipelines typically chain three stages, or collapse them into a single speech-to-speech model:

1. **STT** (speech-to-text) — Whisper, Deepgram, or browser `SpeechRecognition`
2. **LLM reasoning** — the text turn is sent to the model as normal
3. **TTS** (text-to-speech) — ElevenLabs, OpenAI TTS, or on-device synthesis

Newer **realtime** APIs (e.g. OpenAI Realtime, or streaming speech-to-speech models) skip the text intermediate step entirely, streaming audio in and out over a WebSocket/WebRTC connection for sub-second latency — critical for natural-feeling voice assistants.

Code — Streaming mic audio to a realtime voice endpoint over WebSocket

```
// lib/voiceStream.ts
export async function startVoiceSession(onAudioChunk: (chunk: ArrayBuffer) => void) {
  const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
  const ctx = new AudioContext({ sampleRate: 16000 });
  const source = ctx.createMediaStreamSource(stream);
  const processor = ctx.createScriptProcessor(4096, 1, 1);

  const socket = new WebSocket(`${process.env.NEXT_PUBLIC_WS_URL}/voice`);
  socket.binaryType = 'arraybuffer';
  socket.onmessage = (evt) => onAudioChunk(evt.data); // synthesized reply audio

  processor.onaudioprocess = (e) => {
    const pcm = e.inputBuffer.getChannelData(0);
    const int16 = new Int16Array(pcm.length);
    for (let i = 0; i < pcm.length; i++) int16[i] = pcm[i] * 32767;
    if (socket.readyState === WebSocket.OPEN) socket.send(int16.buffer);
  };

  source.connect(processor);
  processor.connect(ctx.destination);
  return () => { processor.disconnect(); source.disconnect(); socket.close(); };
}
```

08 · DevOps, Deployment & Observability

Version Control & CI

- **Git** — distributed version control; feature-branch + PR workflow is standard.
- **GitHub / GitLab** — hosting, code review, and CI/CD pipelines (GitHub Actions, GitLab CI). Claude Code can operate directly against a GitHub/GitLab remote for automated PRs.

Deployment

- **Vercel** — zero-config deploys for Next.js and most frontend frameworks, with edge functions and preview URLs per pull request.
- **Supabase** — hosted Postgres + Auth + Storage + realtime subscriptions + edge functions, effectively a full backend-as-a-service.

Code — Sentry initialization (Node + browser)

```
// src/instrumentation.ts (server)
import * as Sentry from '@sentry/node';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  tracesSampleRate: 0.2,
  environment: process.env.NODE_ENV,
});

// app/instrumentation-client.ts (browser, Next.js)
import * as Sentry from '@sentry/nextjs';
Sentry.init({ dsn: process.env.NEXT_PUBLIC_SENTRY_DSN, tracesSampleRate: 0.1 });
```

Code — Supabase client (typed)

```
// src/lib/supabaseClient.ts
import { createClient } from '@supabase/supabase-js';
import type { Database } from '../database.types'; // generated via `supabase gen types`

export const supabase = createClient<Database>(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

// Row-level security means this is safe to call from the client directly
export async function getUserDocs(userId: string) {
  const { data, error } = await supabase
    .from('documents')
    .select('id, title, updated_at')
    .eq('owner_id', userId)
    .order('updated_at', { ascending: false });
  if (error) throw error;
  return data;
}
```

Code — Minimal GitHub Actions CI/CD to Vercel

```
# .github/workflows/deploy.yml
name: CI
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint && npm run typecheck && npm test
      - run: npm run build
```

Observability

- **Sentry.io** — error and performance monitoring across frontend, backend, and edge runtimes; captures stack traces with source maps and release tracking.

Edge Data

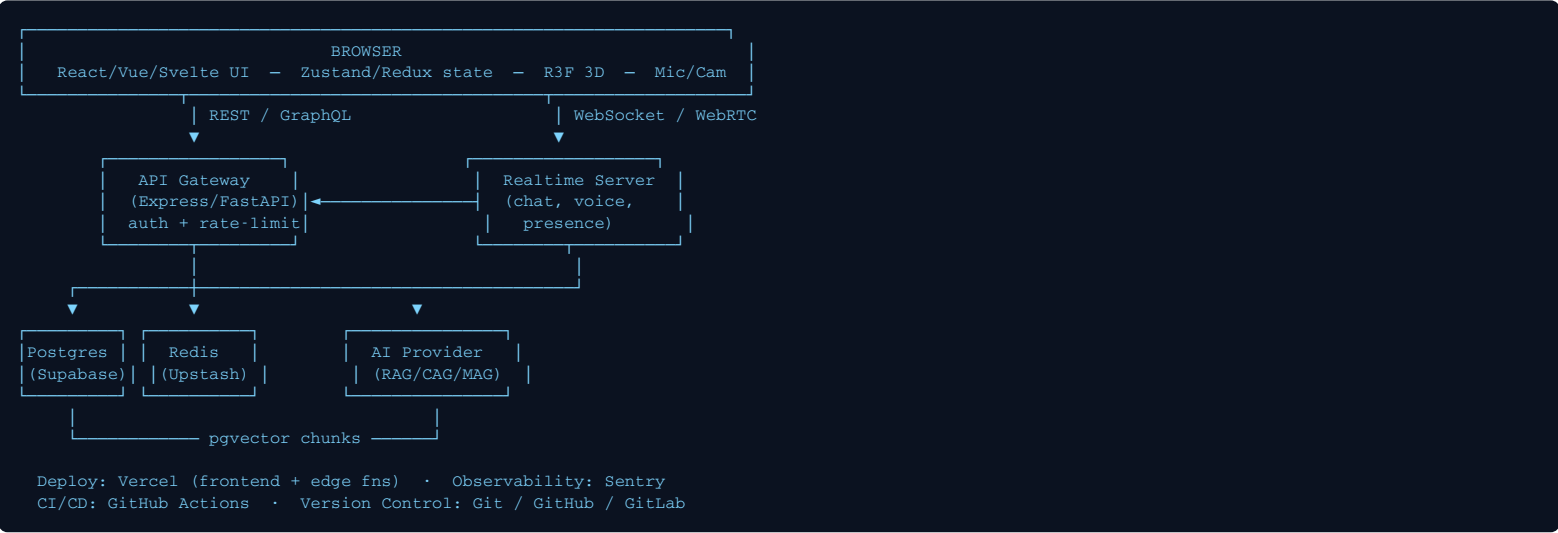
- **Upstash** — serverless Redis and Kafka reachable over HTTP, built for edge/serverless functions that can't hold persistent TCP connections.

09 · API Keys, Tokens & Free Tiers

Service	Free Tier Notes	Key Storage
Anthropic / OpenAI API	Usage-based; small free credits on signup for evaluation	Server-side env var only
Supabase	Generous free project tier (DB + Auth + Storage)	Anon key is public-safe with RLS; service key is server-only
Upstash	Free Redis DB with request-based quota	REST token, server-side
Vercel	Free hobby plan for personal projects	Deployment-scoped env vars via dashboard
Sentry	Free tier with capped event volume	DSN is safe to expose client-side
GitHub	Free private repos + Actions minutes	Fine-grained personal access tokens, scoped & expiring

Golden rule: any key that can spend money or read private data (AI provider keys, service-role DB keys) must never ship in client-side JavaScript. Only publishable/anon keys explicitly designed for the browser belong there.

10 · Reference Architecture



11 • Companion Code Index

The accompanying code bundle contains working, adapt-and-run TypeScript files referenced throughout this guide:

File	Purpose
store/chatStore.ts	Zustand store for streaming chat UI state
routes/chat.router.ts	Express route with SSE streaming, validation, auth, rate limiting
routes/index.ts	Central router registry / protocol map
ws/server.ts	WebSocket server with auth + heartbeat
lib/cache.ts	Cache-aside pattern with Upstash Redis
lib/chunking.ts	Recursive token-aware text chunking for RAG
lib/rag.ts	Embedding ingestion + pgvector similarity retrieval
lib/memory.ts	MAG-style persistent user memory store
middleware/rateLimiter.ts	Sliding-window rate limiting middleware
middleware/auth.ts	JWT verification middleware
components/GestureScene.tsx	React Three Fiber + MediaPipe hand-gesture control
lib/voiceStream.ts	Mic capture → WebSocket voice streaming
lib/supabaseClient.ts	Typed Supabase client + example query
instrumentation.ts	Sentry init for server + browser
.github/workflows/deploy.yml	CI pipeline: lint, typecheck, test, build

Companion code samples are illustrative reference implementations — review, adapt environment variables, and add production-grade error handling before deploying.

12 · Pre-Launch Security Checklist for Vibe-Coded Apps

A practical, prompt-driven checklist for auditing an AI-assisted ("vibe-coded") app before launch. Each item explains why it matters and gives a ready-to-paste prompt for an AI coding assistant to find and fix the issue. Work through the **Start Here** seven first, then the full list, skipping sections that don't apply.

Start here — do these 7 first

These cover the mistakes that most often end in a data leak, a hijacked account, or a surprise bill.

- Keep every API key and secret on the server, never in frontend code
- Use your database public key on the frontend, never the admin key that skips your security rules
- Turn on row-level security for every database table
- Confirm each user can only reach their own records, not just that they are logged in
- Rate-limit your API, especially login and anything that costs you money per call
- Set billing caps and alerts on every paid service
- Use parameterized queries so user input can never run as a command

Secrets and Keys

1 Keep keys and secrets on the server

Why it matters: If an API key lives in code that runs in the browser, anyone can open developer tools and copy it, then spend money on your accounts.

```
Audit my codebase for any API keys, secrets, or tokens exposed to the client or
bundled into frontend code. Move every secret into server-side environment
variables, and make sure the frontend only ever calls my own backend rather
than calling a third-party API with a secret key. Show me each place you found
a problem and how you fixed it.
```

2 Keep secrets out of your Git history

Why it matters: Once a secret is committed, it stays in your Git history even after you delete it, and bots scan public repos for exactly this.

```
Check whether any secrets, API keys, or .env files have ever been committed
to this repository, including in older commits. Make sure .env and all secret
files are in .gitignore. If anything sensitive is already in the history, tell
me exactly what it is and walk me through rotating those keys and removing them.
```

3 Use your public database key on the frontend, not the admin key

Why it matters: A lot of vibe-coded apps accidentally use the admin database key on the client, which skips every security rule you set up, including row-level security.

```
Check which database key my frontend uses. Confirm it is the public or anon key
and never the service role or admin key that bypasses row-level security. If
the admin key is exposed anywhere on the client, move it to the server and
replace the client usage with the public key.
```

Database

4 Turn on row-level security for every table

Why it matters: Without it, anyone who finds your database endpoint can pull every user's data, not only their own.

```
Enable row-level security on every table in my database. For each table, write
policies so users can only read and change their own rows. Do not use allow-all
or USING (true) policies. List every table with the policy you applied, and
flag any table you were unsure about.
```

5 Encrypt sensitive data

Why it matters: If someone gets into your database and everything sits there in plain text, one leak exposes all of it.

```
Review how my app stores sensitive data such as personal information and tokens.
Make sure sensitive fields are encrypted at rest and never stored in a readable
form. Tell me which fields you changed and which ones you recommend I review.
```

6 Enforce authentication on the server

Why it matters: If your rules only live in the frontend, someone can skip them by calling your backend directly.

```
Check that authentication and authorization are enforced on the server for every
protected route and action, not just hidden in the frontend UI. Show me any
endpoint that trusts the client to say who the user is, and fix it so the
server verifies the user on every request.
```

7 Check that each user owns the record they are touching

Why it matters: A logged-in user who changes an ID in a request can often pull up someone else's data if you only check that they are logged in.

```
Audit my app for broken access control. For every endpoint that reads or updates
a record by ID, confirm the code checks that the logged-in user actually owns
that record, not just that they are authenticated. Show me each place and the
ownership check you added.
```

8 Only accept the fields a user is allowed to change

Why it matters: If an update accepts any field, a user can add something like `role=admin` to a normal profile update and promote themselves.

```
Check my create and update endpoints for mass assignment. Make sure each one
only accepts the specific fields a user is allowed to change, and ignores
anything else such as role, is_admin, or account status. Show me which
endpoints you locked down.
```

9 Store session tokens in secure cookies

Why it matters: Tokens kept in `localStorage` can be read by any script on the page, and a stolen one often keeps working forever.

```
Review how my app stores login and session tokens. Move them out of localStorage
and into secure, http-only cookies, and make sure sessions expire after a
reasonable time. Show me what you changed.
```

10 Hash passwords if you built your own login

Why it matters: If you store passwords in a readable form, one database leak exposes every account.

```
If my app stores passwords directly, make sure they are hashed with a strong
modern algorithm such as bcrypt or argon2, and never stored or logged in plain
text. If I am using an auth provider that handles this, confirm it and tell me
I am covered.
```

Rate Limiting and Abuse

11 Rate-limit your API

Why it matters: Without limits, one person can hit your endpoints thousands of times, taking your app down or running up your bill. On login, it also lets them guess passwords as fast as they want.

```
Add rate limiting to my API routes. Apply stricter limits on login, signup,
password reset, and any endpoint that calls a paid service such as an AI model
or email provider. Enforce the limit on the server so it cannot be bypassed
from the client.
```

12 Set billing caps and alerts

Why it matters: A sudden spike on a paid service can become a large bill before you ever notice.

```
Help me set up billing caps and usage alerts on the paid services my app uses,
so I get notified early instead of finding out from a large invoice. List which
services support this and walk me through configuring each one.
```

13 Add bot protection to public forms

Why it matters: Open signup and public forms get hit by bots creating fake accounts or spamming submissions.

```
Add bot protection to my signup form and any public-facing forms, using a
CAPTCHA or a similar challenge. Make sure it runs on the server before the
request is processed, not only in the browser.
```

🔔 Input and Output

14 Use parameterized queries

Why it matters: If your app builds queries out of raw user input, someone can type commands that run against your database.

```
Audit my database queries for SQL or query injection. Replace any query that
builds a string out of user input with parameterized queries or the safe
methods my database library provides. Show me every query you changed.
```

15 Validate and sanitize user input

Why it matters: Trusting whatever a user submits leads to broken data and opens the door to several kinds of attack.

```
Add server-side validation and sanitization for all user input across my app.
Validate type, length, and format, and reject anything that does not match.
Do this on the server even if the frontend already validates. Show me which
inputs you covered.
```

16 Escape user content before displaying it

Why it matters: If your app shows text a user typed without escaping it, someone can slip in code that runs in other people's browsers and steals their sessions. This is cross-site scripting, a different attack from injection into your database.

```
Check my app for cross-site scripting. Make sure any user-generated content
is escaped or sanitized before it is rendered, and avoid unsafe methods that
inject raw HTML. Show me each spot you fixed.
```

17 Lock down file uploads

Why it matters: If people can upload files and you do not check them, someone can upload a file that runs code on your server.

```
If my app allows file uploads, validate the file type and size on the server,
restrict allowed types to what I actually need, and store uploads somewhere
they cannot be executed. Show me how you secured the upload flow.
```

18 Do not return more data than the screen needs

Why it matters: APIs often hand back whole database records, including fields the screen never uses, like other people's emails or internal flags.

```
Review my API responses for excessive data exposure. Make sure each endpoint
only returns the fields the client actually needs, and never leaks sensitive
fields such as password hashes, tokens, or other users' personal data. Show me
which responses you trimmed.
```

💰 Payments (only if you take money)

19 Verify payment webhook signatures

Why it matters: Your payment provider notifies your app when a payment succeeds, and if you do not verify that message is real, someone can fake it and get your product for free.

```
If my app receives payment webhooks, verify the webhook signature on every one
so only genuine events from my payment provider are accepted. Reject anything
that fails verification. Show me the verification you added.
```

20 Set prices on the server, never from the client

Why it matters: If the price comes from the frontend and your server trusts it, someone can change what they pay before it reaches you.

```
Make sure my app never trusts a price or amount sent from the client. Set and
verify all prices on the server based on the product being purchased, not on
values submitted by the browser. Show me where prices currently come from and
fix any that come from the client.
```

AI Features (only if you use LLMs)

21 Defend against prompt injection and unsafe output

Why it matters: Users can hide instructions inside their input to hijack your AI feature, and blindly running or rendering the model's output can expose your app.

```
Review how my app uses AI models. Add protection against prompt injection by
keeping user input separate from my system instructions so user input cannot
override my prompts. Treat the model output as untrusted, so it is escaped
before display and never executed directly. Show me what you changed.
```

22 Cap AI usage per user

Why it matters: Without per-user limits, one person can loop your AI feature and drain your credits overnight.

```
Add per-user usage limits on any feature that calls a paid AI model, so a
single user cannot make unlimited requests. Enforce the cap on the server and
return a clear message when it is reached. Show me how you applied it.
```

Deployment and Ops

23 Force HTTPS

Why it matters: Without HTTPS, data between your users and your app travels in plain text that anyone on the same network can read.

```
Make sure my app forces HTTPS everywhere and redirects any HTTP traffic to
HTTPS. Confirm my hosting setup enforces this and tell me if anything is still
served insecurely.
```

24 Add security headers

Why it matters: A handful of standard headers block common attacks, such as your site being loaded inside a scam page to trick your own users.

```
Add the standard security headers to my app, including Content-Security-Policy,
X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security.
Use sensible defaults for my stack and explain what each one does.
```

25 Turn off debug mode and hide source maps and .git in production

Why it matters: Debug mode, exposed source maps, and an accessible .git folder can hand an attacker a map of your app and its secrets.

```
Check my production setup for debug mode left on, source maps served publicly,
and the .git folder being accessible. Turn off debug in production and make
sure none of these are exposed. Show me what you found.
```

26 Keep secrets out of error messages

Why it matters: Detailed errors can leak stack traces, file paths, and secrets straight to whoever triggered them.

```
Review how my app handles errors in production. Make sure users see a generic
error message while the full details are logged privately, so stack traces and
secrets are never shown to the client. Show me what you changed.
```

27 Keep your dependencies updated

Why it matters: A lot of apps get breached through a known hole in an out-of-date package rather than anything custom.

Scan my dependencies for known vulnerabilities and list anything outdated or flagged. Update the ones that are safe to update, and tell me about any that need manual attention or could be a breaking change.

28 Turn on logging and monitoring

Why it matters: Without logs you will not know an attack happened, and careless logs can leak secrets themselves.

Set up basic logging and monitoring so I can see errors and suspicious activity. Make sure passwords, tokens, and other secrets are never written to the logs. Show me what you added and where the logs go.

29 Set up automatic backups

Why it matters: If your data gets wiped or corrupted and you have no backup, it is gone for good.

Help me set up automatic backups for my database on a regular schedule, and confirm I can actually restore from them. Walk me through the setup for my hosting or database provider.

30 Turn on two-factor for your own accounts

Why it matters: Attackers often get in through your hosting, database, or domain account rather than your code.

Give me a short checklist of the accounts most worth protecting with two-factor authentication, including my hosting, database, domain registrar, and email, so I can turn it on for each one.

Optional — Building a Mobile App?

M1 Keep API keys out of your app bundle

Why it matters: Anyone can unpack a mobile app and read the strings inside it, so a key shipped in the bundle is effectively public.

This is a mobile app. Check for any API keys or secrets bundled into the app or its JavaScript bundle. Move them server-side and have the app call my backend instead of holding secrets directly.

M2 Store tokens in secure storage, not AsyncStorage

Why it matters: AsyncStorage is not encrypted, so a login token kept there can be read off the device.

Check where my mobile app stores auth tokens. Move them out of AsyncStorage and into the platform secure storage, such as Keychain on iOS or Keystore on Android.

M3 Validate deep links

Why it matters: A deep link that triggers actions without checks can be abused to make your app do things the user never intended.

Review how my app handles deep links, and make sure any action triggered by a link validates the user and the request before doing anything sensitive.

M4 Do not rely on biometrics alone for sensitive actions

Why it matters: A biometric check on the device can be bypassed, so it should gate the interface, not stand in for a real server-side check.

Confirm that biometric authentication in my app only controls local access, and that sensitive actions are still verified on the server rather than trusting the device.

How to use this checklist: Run through the Start Here seven before anything else. Then work down the full list, skipping the Payments, AI, and Mobile sections if they don't apply to your app. Paste each prompt into your AI coding assistant one at a time rather than all at once, so you can review what it changes on each pass. Come back to the whole checklist after any major feature update, since new features tend to open new holes.

13 · From Vibe-Coded to Production-Grade

Moving from a "vibe-coded" project to a production-grade application is a shift from pure aesthetics to user experience, reliability, and edge-case handling. Vibe coding gets the happy path working and makes it look cool; professional engineering ensures the app doesn't fall apart when the internet is slow or the user does something unexpected. This section is an exhaustive breakdown of what separates a weekend project from a polished, enterprise-ready application.

13.1 Visual & UI Restraint (Maturing the Aesthetics)

Ditch the neon & heavy glows.

"Vibe coded" apps often rely on heavy `box-shadow` glows and stark, high-contrast neon borders. Professional apps use subtle, layered shadows to create depth (elevation) and `1px` borders with low opacity (e.g. `rgba(255,255,255,0.1)`) to define boundaries without screaming for attention.

The "squircle" and border radii.

Inconsistent border radiuses make a site look messy. Define a strict token system (e.g. `sm: 4px` , `md: 8px` , `lg: 12px`) and stick to it. Inner elements should have a slightly smaller border-radius than their container to look mathematically correct.

Color palette discipline.

Stop using raw hex codes everywhere. Establish a primary brand color, a semantic system (success, warning, error, info), and a neutral greyscale (from background darks to text lights). Ensure your text-to-background contrast meets WCAG accessibility standards.

Typographic hierarchy.

A real app doesn't just use one font size. Establish a clear scale (H1 through H6, body text, small text). Use professional geometric or neo-grotesque sans-serifs (like Inter, Geist, or SF Pro) and lower the opacity of secondary text (like timestamps or descriptions) to create visual relief.

13.2 State Management & Perceived Performance

Skeletal loaders & Suspense.

Instead of a jarring full-page loading spinner, use skeleton screens that mimic the layout of the incoming data. In React, pairing `<Suspense>` boundaries with these skeletons makes the data-fetching waterfall feel intentional and smooth.

Optimistic UI updates.

When a user clicks "Like" or submits a form, don't wait for the database response to update the UI. Immediately show the success state locally, and silently roll it back if the server request fails. This makes the app feel instantly responsive.

Graceful degradation of images.

Don't let images pop in abruptly or break the layout. Use a blur-hash or a solid low-contrast background color as a placeholder while the image loads. Prevent Cumulative Layout Shift (CLS) by explicitly defining image dimensions.

Empty states.

What happens when a user has no data, no messages, or a search returns zero results? Vibe-coded apps show a blank screen or a raw JSON array. Professional apps have beautifully designed "empty state" components with clear calls to action (e.g. "You have no projects yet. Click here to create your first one.").

Code — Optimistic UI update with rollback (React)

```
// hooks/useOptimisticLike.ts
import { useState } from 'react';

export function useOptimisticLike(postId: string, initialLiked: boolean) {
  const [liked, setLiked] = useState(initialLiked);

  async function toggleLike() {
    const previous = liked;
    setLiked(!previous); // optimistic update, instant feedback

    try {
      await fetch(`/api/v1/posts/${postId}/like`, { method: 'POST' });
    } catch {
      setLiked(previous); // silent rollback on failure
    }
  }

  return { liked, toggleLike };
}
```

13.3 Feedback, Errors & Resilience

Nuanced error handling.

Never show a raw backend error message to a user. If a fetch fails, provide a user-friendly toast notification. Wrap frontend components in Error Boundaries so that if one widget breaks, it doesn't crash the entire page — it just displays a localized fallback UI.

Micro-interactions & focus states.

Every clickable element needs a hover state and an active/click state (e.g. scaling down slightly to `0.98`). More importantly, professional apps respect keyboard navigation — use custom, highly visible `:focus-visible` rings so power users can tab through the UI effortlessly.

Strict form validation.

Validate inputs on the client side instantly (email formats, password strength, numeric limits) before the user even clicks submit. Show the error directly beneath the input field, not in a generic alert box.

Disabled states.

Buttons and inputs should be clearly disabled (lowered opacity, `cursor-not-allowed`) when a form is submitting, to prevent duplicate API calls.

Code — React error boundary with localized fallback

```
// components/ErrorBoundary.tsx
import { Component, ReactNode } from 'react';
import * as Sentry from '@sentry/react';

interface Props { children: ReactNode; fallback: ReactNode; }
interface State { hasError: boolean; }

export class ErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error: Error, info: React.ErrorInfo) {
    Sentry.captureException(error, { extra: { componentStack: info.componentStack } });
  }

  render() {
    if (this.state.hasError) return this.props.fallback;
    return this.props.children;
  }
}

// Usage: wrap a widget, not the whole app, so one failure stays localized
// <ErrorBoundary fallback={<WidgetErrorCard />>
//   <RecommendationsWidget />
// </ErrorBoundary>
```

13.4 Code Architecture & Infrastructure

Application monitoring.

Professional developers don't rely on users to tell them the site is broken. Integrate error-tracking telemetry (like Sentry) to catch unhandled exceptions, latency issues, and broken API routes the moment they happen in production.

Strict typing.

If you are passing data between your frontend and backend, use TypeScript to define those contracts. Knowing exactly what shape your data is in prevents the classic `undefined is not a function` crashes that plague amateur apps.

Modular architecture.

Stop putting 1,000 lines of code in one file. Break your UI down into atomic, reusable components (buttons, inputs, cards). It ensures visual consistency across the entire platform.

Environment parity.

Your staging environment should perfectly mirror your production environment. Secrets, environment variables, and database connection strings should be strictly managed, never hardcoded.

13.5 Intent, Predictability & State

Core principle — predictability. Users should never have to guess what an interaction will do before they commit to it. If a button is just an icon, a crisp, zero-delay tooltip explaining its exact function is mandatory.

URL-driven state.

Vibe-coded apps handle modal opens, search filters, and tab switches in local component state. Professional apps push this to the URL (e.g. `?tab=billing` or `?modal=edit-profile`). This lets users hit "Refresh," use the browser's back button, or send a link to a coworker, and the app loads in the exact same state.

Draft persistence.

If a user writes three paragraphs in a form and accidentally hits refresh or closes the tab, a pro app restores that text from `localStorage` or `sessionStorage` when they return. Vibe apps wipe the data and force them to start over.

Idempotency & double-click prevention.

Vibe apps let you click "Submit Payment" five times if the server is lagging. Pro apps instantly disable the button on the first click, swap the text/icon to a loading state, and ensure the backend only processes one request.

Code — URL-driven modal state (React Router / Next.js searchParams)

```
// hooks/useUrlModal.ts
import { useSearchParams } from 'next/navigation';
import { useRouter, usePathname } from 'next/navigation';

export function useUrlModal(key = 'modal') {
  const params = useSearchParams();
  const router = useRouter();
  const pathname = usePathname();

  const open = params.get(key);

  function openModal(name: string) {
    const next = new URLSearchParams(params);
    next.set(key, name);
    router.push(`${pathname}?${next.toString()}`);
  }

  function closeModal() {
    const next = new URLSearchParams(params);
    next.delete(key);
    router.push(`${pathname}?${next.toString()}`);
  }

  return { open, openModal, closeModal };
}
```

Code — Idempotent submit-button guard

```
// hooks/useIdempotentSubmit.ts
import { useRef, useState } from 'react';

export function useIdempotentSubmit<T>(action: () => Promise<T>) {
  const [pending, setPending] = useState(false);
  const inFlight = useRef(false);

  async function submit() {
    if (inFlight.current) return; // hard guard against double-click races
    inFlight.current = true;
    setPending(true);
    try {
      return await action();
    } finally {
      inFlight.current = false;
      setPending(false);
    }
  }

  return { submit, pending };
}
```

13.6 Advanced Interaction & Power-User Features

Debouncing & throttling API calls.

Vibe apps send a search API request for every single keystroke. Pro apps debounce — waiting about 300ms after the user *stops* typing before firing the request. It stops the UI from stuttering and saves your server infrastructure from unnecessary load.

Meaningful fallbacks.

Broken image icons are a dead giveaway of an unpolished app. If a user's avatar fails to load, silently catch the error and display a fallback — like their initials centered on a deterministic background color generated from their username.

Keyboard hotkeys.

Beyond basic `Tab` indexing, pro apps anticipate power users: `Cmd/Ctrl+K` to open a global search palette, `Escape` to close any open dropdown or modal instantly, and `Enter` to submit forms from any input field within that form.

Code — Debounced search hook

```
// hooks/useDebounceSearch.ts
import { useEffect, useState } from 'react';

export function useDebounceValue<T>(value: T, delayMs = 300): T {
  const [debounced, setDebounce] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounce(value), delayMs);
    return () => clearTimeout(timer);
  }, [value, delayMs]);

  return debounced;
}

// Usage:
// const [query, setQuery] = useState('');
// const debouncedQuery = useDebounceValue(query, 300);
// useEffect(() => { if (debouncedQuery) runSearch(debouncedQuery); }, [debouncedQuery]);
```

Code — Global hotkey handler (Cmd/Ctrl+K, Escape)

```
// hooks/useGlobalHotkeys.ts
import { useEffect } from 'react';

export function useGlobalHotkeys({
  onCommandK,
  onEscape,
}): {
  onCommandK?: () => void;
  onEscape?: () => void;
}) {
  useEffect(() => {
    function handler(e: KeyboardEvent) {
      const isCmdK = (e.metaKey || e.ctrlKey) && e.key.toLowerCase() === 'k';
      if (isCmdK) {
        e.preventDefault();
        onCommandK?.();
      }
      if (e.key === 'Escape') onEscape?.();
    }
    window.addEventListener('keydown', handler);
    return () => window.removeEventListener('keydown', handler);
  }, [onCommandK, onEscape]);
}
```

13.7 The "Unseen" Visual Polish

Scrollbar gutters & layout jumps.

When you open a modal, the background scrollbar usually disappears, causing the entire website layout to violently jerk sideways. Using CSS `scrollbar-gutter: stable` or properly managing `padding-right` on the body prevents this jarring layout shift.

Fluid typography and spacing.

Vibe apps look great on an iPhone 14 and a 1080p monitor but break on an iPad or a Galaxy Fold. Pro apps use CSS `clamp()` for typography and spacing so elements smoothly scale across any viewport size, rather than snapping awkwardly at hard breakpoints.

Intelligent copywriting.

Instead of a generic "Submit," use action-oriented verbs like "Save Changes" or "Deploy Project." Instead of a raw "Error 500," use human-readable error states like "We couldn't connect to the server. Please check your connection and try again."

Code — Fluid type scale with clamp()

```

/* tokens.css */
:root {
  --font-h1: clamp(1.75rem, 1.2rem + 2.2vw, 3rem);
  --font-h2: clamp(1.4rem, 1.1rem + 1.4vw, 2.1rem);
  --font-body: clamp(0.95rem, 0.9rem + 0.2vw, 1.05rem);
  --space-section: clamp(2rem, 1.5rem + 2vw, 5rem);

  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 12px;

  --border-subtle: rgba(255, 255, 255, 0.1);
  --shadow-elevation-1: 0 1px 2px rgba(0,0,0,0.06), 0 1px 1px rgba(0,0,0,0.04);
  --shadow-elevation-2: 0 4px 12px rgba(0,0,0,0.08), 0 2px 4px rgba(0,0,0,0.04);
}

h1 { font-size: var(--font-h1); }
h2 { font-size: var(--font-h2); }
body { font-size: var(--font-body); }
.card { border-radius: var(--radius-md); box-shadow: var(--shadow-elevation-1); }

```

Code — Deterministic avatar fallback (initials + color)

```

// lib/avatarFallback.ts
export function initialsAvatar(name: string) {
  const initials = name
    .split(' ')
    .map((p) => p[0])
    .slice(0, 2)
    .join('')
    .toUpperCase();

  // Deterministic hue from the username, so the same user always gets the same color
  let hash = 0;
  for (let i = 0; i < name.length; i++) hash = name.charCodeAt(i) + ((hash << 5) - hash);
  const hue = Math.abs(hash) % 360;

  return { initials, background: `hsl(${hue}, 55%, 45%)` };
}

// <img src={user.avatarUrl} onError={showFallback} />
// On error, render a <div> with initialsAvatar(user.name) instead

```

End of guide. All code samples and prompts throughout this guide are reference material — review each change an AI assistant makes before deploying it, and treat the security checklist and polish checklist as a floor, not a ceiling, for a production-grade release.

14 · Pre-Launch Website & Content Checklist

Twenty items that get missed even on otherwise well-engineered apps — the SEO, legal, and conversion basics that determine whether a page gets found, trusted, and used. Run through this alongside the security and production-polish checklists before going live.

🔍 SEO & Discoverability

3

Meta title for every page

Why it matters: The title tag is what shows in search results and browser tabs. A missing or generic title (or every page sharing one) tanks click-through rate and search ranking.

```
<title>Pricing — Acme Analytics</title>
<!-- Keep under ~60 characters, unique per page, primary keyword near the front -->
```

4

Meta description for every page

Why it matters: The description is the preview snippet under your title in search results. Without one, Google pulls a random chunk of page text, which often reads badly and hurts click-through.

```
<meta name="description" content="Acme Analytics turns raw event data into
dashboards your whole team can read. Start free, no credit card required." />
<!-- ~150-160 characters, unique per page, includes a reason to click -->
```

5

Open Graph image

Why it matters: Without an OG image, links shared on Slack, X, LinkedIn, or iMessage render as a bare, untrustworthy-looking text link instead of a rich preview card.

```
<meta property="og:title" content="Acme Analytics" />
<meta property="og:description" content="Dashboards your whole team can read." />
<meta property="og:image" content="https://acme.com/og-image.png" />
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
<meta name="twitter:card" content="summary_large_image" />
```

6

Favicon

Why it matters: A missing favicon shows a broken-image icon in browser tabs and bookmarks — a small but immediate signal that the site is unfinished.

```
<link rel="icon" href="/favicon.ico" sizes="any" />
<link rel="icon" href="/icon.svg" type="image/svg+xml" />
<link rel="apple-touch-icon" href="/apple-touch-icon.png" />
```

7

robots.txt

Why it matters: Without it, you have no control over which parts of your site crawlers index — and a misconfigured one left over from staging can accidentally block search engines from your entire production site.

```
# /robots.txt
User-agent: *
Allow: /
Disallow: /admin
Disallow: /api

Sitemap: https://acme.com/sitemap.xml
```

8 sitemap.xml

Why it matters: A sitemap tells search engines every page that exists and how important each one is, which speeds up indexing — especially for pages with few internal links pointing to them.

```
<!-- /sitemap.xml -->
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://acme.com/</loc>
    <changefreq>weekly</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://acme.com/pricing</loc>
    <changefreq>monthly</changefreq>
    <priority>0.8</priority>
  </url>
</urlset>
<!-- Next.js: generate this dynamically via app/sitemap.ts instead of hand-writing it -->
```

9 Alt text on every image

Why it matters: Missing alt text breaks the experience for screen-reader users, fails basic accessibility audits, and gives search engines nothing to index the image by.

```

<!-- Decorative-only images: alt="" so screen readers skip them, not omitted entirely -->
```

🕒 UX & Conversion

1 Custom 404 page

Why it matters: A default server error page is a dead end that loses visitors. A custom 404 keeps them on-site with navigation back to something useful.

Design a custom 404 page that matches my site's branding, includes a short friendly message, a search bar or link back to the homepage, and links to my most important pages. Show me the component you created.

2 CTA above the fold

Why it matters: Most visitors never scroll. If the primary call-to-action is buried below the first screen, you lose conversions from people who would have clicked if they'd seen it.

Review my landing page and confirm the primary call-to-action button is visible without scrolling on both desktop and mobile viewports. If it isn't, restructure the hero section so it is.

10 Mobile breakpoints

Why it matters: Most traffic is mobile. A layout only tested on a desktop monitor often breaks — overflowing text, unreachable buttons, horizontal scroll — the moment it's viewed on a phone.

Test my site at common breakpoints (375px, 768px, 1024px, 1440px). Fix any overflow, unreadable text, overlapping elements, or unreachable tap targets at each size. Show me what you found and fixed.

11 Sticky mobile CTA

Why it matters: On mobile, a CTA that scrolls away with the page is easy to lose. A sticky bottom bar keeps the action reachable at all times without the user hunting for it.

Add a sticky call-to-action bar that stays fixed to the bottom of the screen on mobile viewports only, doesn't obstruct content, and disappears when a modal or footer is in view.

12 Loading states

Why it matters: An unresponsive-looking screen during a fetch reads as broken. Covered in more depth in Section 13.2 (skeletal loaders, Suspense) — confirm every async action has one.

Audit every page and component that fetches data or submits a request. Confirm each one shows a loading state (skeleton, spinner, or disabled button) rather than appearing frozen or unresponsive. List anything missing one.

13 Form error states

Why it matters: A form that fails silently, or shows one generic error for every field, leaves users guessing what to fix. Covered further in Section 13.3 — confirm every form field has its own inline error.

Check every form on my site. Confirm each field shows its own specific, inline error message beneath it when invalid, rather than one generic banner at the top of the form.

14 Thank-you page

Why it matters: Without a clear confirmation after signup, purchase, or form submission, users aren't sure the action worked — and you lose the chance to set next-step expectations or fire a conversion event.

Add a dedicated thank-you or confirmation page after signup, purchase, and contact-form submission. Confirm it fires my conversion tracking event and tells the user what happens next.

Legal & Compliance

15 Privacy policy

Why it matters: If you collect any personal data — even just an email for signup or analytics — most jurisdictions require you to disclose what you collect and why. Missing one is also an instant trust red flag.

Draft a privacy policy for my app covering what data I collect, why, how long I keep it, and which third-party services (analytics, payments, email) I share it with. Link it in my footer and at every signup/data-collection point.

16 Terms & conditions

Why it matters: Terms set the rules of use, limit your liability, and give you a basis to suspend abusive accounts. Without them, disputes default entirely to whatever a court decides is reasonable.

Draft terms and conditions covering acceptable use, account termination, liability limitations, and payment terms if applicable. Link it in my footer and require acceptance at signup.

17 Cookie banner

Why it matters: If you use analytics, ads, or any non-essential cookies, regulations like GDPR and ePrivacy require informed consent before those cookies are set, not just a passive notice.

Add a cookie consent banner that blocks non-essential cookies (analytics, ads) from loading until the user consents, offers a clear reject option, and lets users change their choice later. Confirm essential cookies aren't blocked.

19 Real contact information

Why it matters: A site with no visible way to reach a real person reads as untrustworthy or even scammy, and in some jurisdictions a physical address or business registration is a legal requirement.

Add a real, monitored contact method — email, contact form, or support chat — and make sure it's reachable from the footer on every page, not just a dedicated contact page.

Trust & Performance

18 Analytics installed

Why it matters: Without analytics, you're launching blind — no visibility into what pages people actually visit, where they drop off, or whether your CTA is even being clicked.

```
Install analytics (such as Plausible, PostHog, or GA4) across every page, and
confirm pageviews and key conversion events (signup, purchase, form submit)
are firing correctly. Make sure it respects the cookie consent choice above.
```

20 Compressed images

Why it matters: Uncompressed, full-resolution images are one of the most common causes of a slow site — they hurt load time, Core Web Vitals, and mobile data usage.

```
Audit every image on my site for file size. Convert to modern formats (WebP
or AVIF), compress without visible quality loss, and serve responsive sizes
via srcset so mobile doesn't download a desktop-sized image. Show me the
before/after sizes.
```

End of guide. All code samples, prompts, and checklists throughout this guide are reference material — review each change an AI assistant makes before deploying it, and treat these checklists as a floor, not a ceiling, for a production-grade release.

15 · Beyond the Checklist: Deeper Production Maturity

Code quality, security, UI polish, and SEO cover a lot of ground — but they're not the whole picture. This section fills in the parts of "production-grade" that a checklist alone tends to miss: verifying the app actually works (testing), works for everyone (accessibility depth), survives real load (scalability), evolves safely (database lifecycle), is diagnosable in the dark (observability), doesn't quietly bankrupt you (cost management), and holds up as more than one person touches the code (team process and compliance).

15.1 Testing — The Biggest Gap

A vibe-coded app with zero test coverage is one of the most common reasons it breaks in production. If your CI pipeline runs `npm test` but no tests exist, that step is silently doing nothing.

Unit testing.

Tests a single function or component in isolation — pure logic, formatters, validators, reducers. Fast to run, cheap to write, and the first line of defense against regressions. **Vitest** (native to Vite/modern TS projects) or **Jest** are the standard choices.

Integration testing.

Tests how multiple units work together — an API route hitting a real (or test) database, a form submitting through actual validation and state logic. Catches the bugs unit tests miss because the pieces work fine alone but break when wired together.

End-to-end (E2E) testing.

Drives a real browser through a real user flow — sign up, add an item, check out. Slowest and most brittle tier, so reserve it for critical paths rather than every possible click. **Playwright** is the current standard; **Cypress** is a common alternative.

Code — Unit test (Vitest)

```
// lib/chunking.test.ts
import { describe, it, expect } from 'vitest';
import { chunkText } from './chunking';

describe('chunkText', () => {
  it('splits long text into chunks under the token limit', () => {
    const longText = 'Sentence one. '.repeat(200);
    const chunks = chunkText(longText, { maxTokens: 100, overlapTokens: 20 });

    expect(chunks.length).toBeGreaterThan(1);
    chunks.forEach((c) => expect(c.tokens).toBeLessThanOrEqual(120)); // allow overlap slack
  });

  it('returns a single chunk for short text', () => {
    const chunks = chunkText('A short sentence.', { maxTokens: 400 });
    expect(chunks).toHaveLength(1);
  });

  it('carries overlap text forward between chunks', () => {
    const chunks = chunkText('A. '.repeat(500), { maxTokens: 50, overlapTokens: 10 });
    expect(chunks[1].text.startsWith(chunks[0].text.slice(-5))).toBe(false); // sanity, not exact match
    expect(chunks.length).toBeGreaterThan(2);
  });
});
```

Code — Integration test (API route + test database)


```
// routes/user.router.test.ts
import { describe, it, expect, beforeEach } from 'vitest';
import request from 'supertest';
import { app } from '../app';
import { resetTestDb, seedUser } from '../test/utils';

describe('GET /v1/users/:id', () => {
  beforeEach(async () => { await resetTestDb(); });

  it('returns the user for an authenticated owner', async () => {
    const { user, token } = await seedUser();
    const res = await request(app)
      .get(`/v1/users/${user.id}`)
      .set('Authorization', `Bearer ${token}`);

    expect(res.status).toBe(200);
    expect(res.body.id).toBe(user.id);
  });

  it('returns 401 with no token', async () => {
    const { user } = await seedUser();
    const res = await request(app).get(`/v1/users/${user.id}`);
    expect(res.status).toBe(401);
  });
});
```

Code — E2E test (Playwright)

```
// e2e/signup.spec.ts
import { test, expect } from '@playwright/test';

test('user can sign up and reach the dashboard', async ({ page }) => {
  await page.goto('/signup');
  await page.getByLabel('Email').fill('new.user@example.com');
  await page.getByLabel('Password').fill('a-strong-password-123');
  await page.getByRole('button', { name: 'Create account' }).click();

  await expect(page).toHaveURL(/\/dashboard/);
  await expect(page.getByText('Welcome')).toBeVisible();
});

test('signup shows inline error for a weak password', async ({ page }) => {
  await page.goto('/signup');
  await page.getByLabel('Password').fill('123');
  await page.getByRole('button', { name: 'Create account' }).click();

  await expect(page.getByText(/password must be at least/i)).toBeVisible();
});
```

Code — CI updated to run all three tiers

```
# .github/workflows/deploy.yml (excerpt)
- run: npm run test:unit # vitest — fast, runs on every push
- run: npm run test:integration # spins up a test DB via docker-compose
- run: npx playwright install --with-deps && npm run test:e2e
```

15.2 Accessibility — Beyond the Basics

Contrast ratios, alt text, and focus rings (covered in Sections 1 and 13) are necessary but not sufficient. Real accessibility means the app is actually usable by someone who can't see the screen or can't use a mouse — which only surfaces through deliberate testing.

Screen-reader testing.

Run your key flows with VoiceOver (macOS/iOS, built-in), NVDA (Windows, free), or a browser extension audit tool. Confirm form labels are announced, error messages are read aloud when they appear, and nothing important is conveyed by color or icon alone.

ARIA patterns beyond the basics.

Custom components (modals, comboboxes, tabs, toasts) need explicit roles and state — `role="dialog"` with `aria-modal` and focus trapping, `aria-live` regions for async status updates, `aria-expanded` on disclosure triggers. Follow the [ARIA Authoring Practices Guide](#) patterns rather than inventing your own.

Keyboard-only flows, end to end.

Unplug the mouse and try to complete signup, checkout, and your core feature using only Tab, Shift+Tab, Enter, and Escape. Anything unreachable, any focus trap you can't escape, or any action with no keyboard equivalent is a real blocker, not an edge case.

Code — Accessible modal with focus trap and live region

```
// components/AccessibleModal.tsx
import { useEffect, useRef } from 'react';

export function AccessibleModal({
  open,
  onClose,
  title,
  children,
}): {
  open: boolean;
  onClose: () => void;
  title: string;
  children: React.ReactNode;
}) {
  const dialogRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    if (!open) return;
    const previouslyFocused = document.activeElement as HTMLElement;
    dialogRef.current?.focus();

    function onKeyDown(e: KeyboardEvent) {
      if (e.key === 'Escape') onClose();
    }
    window.addEventListener('keydown', onKeyDown);
    return () => {
      window.removeEventListener('keydown', onKeyDown);
      previouslyFocused?.focus(); // return focus to the trigger on close
    };
  }, [open, onClose]);

  if (!open) return null;

  return (
    <div
      ref={dialogRef}
      role="dialog"
      aria-modal="true"
      aria-labelledby="modal-title"
      tabIndex=-1>
      <h2 id="modal-title">{title}</h2>
      {children}
      <div role="status" aria-live="polite" className="sr-only" />
    </div>
  );
}
```

15.3 Scalability Under Real Load

Load testing.

Don't discover your app's breaking point from real users. Tools like **k6** or **Artillery** simulate concurrent traffic against staging so you find the ceiling — and the error mode — before launch.

Horizontal scaling & statelessness.

API servers should hold no in-memory session or request state that would break if a second instance were added behind a load balancer. Session data belongs in Redis/Postgres, not a process-local variable.

Connection pooling limits.

Serverless functions can each open their own DB connection and exhaust Postgres's connection limit in minutes under load. Use a pooler (PgBouncer, or your provider's built-in pooling, e.g. Supabase's connection pooler) between your app and the database.

Database indexing strategy.

A query that's fast with 100 test rows can fall over at 100,000. Index every column used in a `WHERE`, `JOIN`, or `ORDER BY` on a large table, and periodically review slow-query logs rather than guessing.

CDN/caching strategy beyond Redis.

Static assets, images, and cacheable API responses should sit behind a CDN edge cache (Vercel's edge network, Cloudflare) so requests are served near the user and never touch your origin server at all.

Code — Basic load test (k6)

```
// load/chat-endpoint.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 20 }, // ramp up to 20 virtual users
    { duration: '1m', target: 100 }, // hold at 100
    { duration: '30s', target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<800'], // 95% of requests under 800ms
    http_req_failed: ['rate<0.01'], // less than 1% failures
  },
};

export default function () {
  const res = http.post(
    'https://staging.acme.com/api/v1/chat',
    JSON.stringify({ message: 'What is your refund policy?' }),
    { headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${__ENV.TEST_TOKEN}` } }
  );
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Code — Indexing a large, frequently-filtered table

```
-- migrations/0007_index_orders_lookup.sql
CREATE INDEX CONCURRENTLY idx_orders_user_id_created_at
  ON orders (user_id, created_at DESC);

-- Composite index matches the common query shape:
-- SELECT * FROM orders WHERE user_id = $1 ORDER BY created_at DESC LIMIT 20;
-- CONCURRENTLY avoids locking the table during index creation on a live DB.
```

15.4 Database Lifecycle

Migrations tooling.

Hand-editing a production schema, or applying SQL changes ad hoc, is how staging and production silently drift apart. Use **Prisma Migrate**, **Drizzle**, or **Flyway** so every schema change is a versioned, reviewable file that runs the same way in every environment.

Schema versioning & rollback strategy.

Every migration should have a known way back — either an explicit down-migration or a documented manual rollback. Test the rollback path before you need it under pressure, not during an incident.

Code — Prisma migration workflow

```
# Author a schema change, then generate a versioned migration file
npx prisma migrate dev --name add_orders_index

# Apply pending migrations in CI/production (non-interactive, no prompt)
npx prisma migrate deploy

# schema.prisma
model Order {
  id          String    @id @default(uuid())
  userId      String
  createdAt   DateTime  @default(now())

  @@index([userId, createdAt])
}
```

Code — Drizzle equivalent

```
// drizzle/schema.ts
import { pgTable, uuid, timestamp, index } from 'drizzle-orm/pg-core';

export const orders = pgTable(
  'orders',
  {
    id: uuid('id').defaultRandom().primaryKey(),
    userId: uuid('user_id').notNull(),
    createdAt: timestamp('created_at').defaultNow(),
  },
  (table) => ({
    userCreatedIdx: index('idx_orders_user_created').on(table.userId, table.createdAt),
  })
);

// npx drizzle-kit generate:pg -> writes a versioned SQL migration file
// npx drizzle-kit push:pg -> applies it
```

15.5 Observability Depth

Sentry (Section 8) catches unhandled exceptions, but that's one piece of the picture.

Structured logging.

`console.log` output is unsearchable at scale. Structured (JSON) logs with consistent fields — request ID, user ID, route, duration — let you filter and correlate events instead of scrolling through raw text.

Log aggregation.

Ship logs somewhere centralized and searchable (Better Stack, Datadog, Axiom) rather than leaving them scattered across individual server/container instances that rotate away.

Uptime monitoring & alerting.

An external check (Better Uptime, UptimeRobot, Pingdom) that hits your app from outside your own infrastructure and pages you the moment it's down — because your own monitoring can't tell you your server is unreachable if the server itself is what's down.

SLOs (service-level objectives).

A concrete, written target — "99.5% of requests succeed," "p95 latency under 500ms" — turns "is the app healthy?" from a vibe into a number you can alert on and report against.

Code — Structured logger (pino)

```
// lib/logger.ts
import pino from 'pino';

export const logger = pino({
  level: process.env.LOG_LEVEL ?? 'info',
  redact: ['req.headers.authorization', 'password', 'token'], // never log secrets
  formatters: { level: (label) => ({ level: label }) },
});

// Usage in a request handler:
// logger.info({ requestId, userId, route: '/v1/chat', durationMs }, 'chat_request_completed');
// Every log line is now JSON: filterable by requestId, userId, route, etc.
```

15.6 Cost Management (FinOps)

"Set billing caps" (Section 12, item 12) is the floor. Real cost control means knowing where the money goes before the invoice arrives.

Per-feature AI token cost tracking.

Not all AI calls are equal — a chat feature and a background summarization job can have wildly different cost profiles. Tag and log token usage per feature so you know which one to optimize when the bill grows, instead of debugging blind.

Autoscaling cost tradeoffs.

Autoscaling protects uptime but not your wallet — a traffic spike (or a bug causing a retry loop) scales your compute cost right along with it. Set a maximum instance/concurrency ceiling, not just a minimum.

Code — Per-feature token cost logging

```
// lib/aiCostTracking.ts
import { logger } from './logger';

const PRICE_PER_1K_INPUT = 0.003; // adjust to your model's actual pricing
const PRICE_PER_1K_OUTPUT = 0.015;

export function logAiCost(feature: string, usage: { inputTokens: number; outputTokens: number }) {
  const cost =
    (usage.inputTokens / 1000) * PRICE_PER_1K_INPUT +
    (usage.outputTokens / 1000) * PRICE_PER_1K_OUTPUT;

  logger.info({ feature, ...usage, estimatedCostUsd: Number(cost.toFixed(5)) }, 'ai_usage');
  // Aggregate these in your log platform, grouped by `feature`, to see cost per capability
}
```

15.7 Team & Process Maturity

Code-level fixes only hold up if the process around them does too — this matters as soon as more than one person (including an AI assistant) touches the codebase.

Code review norms.

Every change — including AI-generated ones — gets read by a human before merging. A short, written checklist (tests included? secrets checked? migration reversible?) makes review consistent instead of dependent on who happens to be reviewing.

Documentation standards.

A README that explains how to run the project, a CHANGELOG for what shipped, and comments on the genuinely non-obvious decisions — not a comment on every line, but enough that the next person (or the next AI session) doesn't have to reverse-engineer intent.

Feature flags.

Ship code disabled by default and turn it on for a subset of users or yourself first. This decouples deploying from releasing, so a bad change can be switched off instantly without a rollback deploy.

Staged rollouts.

Release to 5% of traffic, watch error rates and key metrics, then widen — rather than flipping a change on for 100% of users simultaneously and finding out about a regression from support tickets.

Incident response & postmortems.

When something breaks in production, a short blameless writeup — what happened, how it was caught, what fixes it — turns an outage into a system improvement instead of just a bad night.

Code — Minimal feature flag check

```
// lib/featureFlags.ts
import { redis } from './cache';

export async function isEnabled(flag: string, userId: string): Promise<boolean> {
  const config = await redis.get<{ enabled: boolean; rolloutPercent: number }>(`flag:${flag}`);
  if (!config?.enabled) return false;
  if (config.rolloutPercent >= 100) return true;

  // Deterministic bucketing: same user always gets the same result for a given flag
  let hash = 0;
  for (let i = 0; i < userId.length; i++) hash = userId.charCodeAt(i) + ((hash << 5) - hash);
  return Math.abs(hash) % 100 < config.rolloutPercent;
}

// Usage: if (await isEnabled('new-checkout-flow', user.id)) { ... }
```

15.8 Compliance Beyond the Basics

A privacy policy and cookie banner (Section 14) cover general-purpose consumer apps. Some products need more, and it's worth knowing when before it becomes a blocker to closing a deal.

SOC 2.

An independent audit of your security controls, commonly requested by B2B customers — especially enterprise ones — before they'll sign a contract. Requires demonstrable access controls, monitoring, and incident response processes over a period of months, not something to bolt on the week before a deal closes.

HIPAA.

Applies if you handle U.S. health data. Requires a signed Business Associate Agreement with every vendor that touches that data (hosting, database, email, analytics), encryption at rest and in transit, and strict audit logging of who accessed what.

Data residency.

Some customers or regulations (notably GDPR for EU users) require data to be stored in a specific geographic region. Confirm your database, backups, and any AI provider's data processing location before assuming a single global region is fine.

End of guide. All code samples, prompts, and checklists throughout this guide are reference material — review each change an AI assistant makes before deploying it, and treat these checklists as a floor, not a ceiling, for a production-grade release.

